Learning Django

Guide B in the **api-drills** repo: a hands-on guide for the Goldman Sachs Software Diagnostics section and beyond. Read Guide A (Web APIs from the Ground Up) and do ex01 and ex02 first; this guide starts where they stop.

How to use this guide

- Sections 1 to 9 teach one idea each and end with a **Practice** box pointing to an exercise in the repo. Do the exercise straight after reading the section.
- Section 10 is the debugging playbook, section 11 the exam-day plan, section 12 points to the schedule in `PLAN.md`.
- Every code example here was run against Django 6.1 before going into this guide. The examples use a **Movie** app, so they teach the pattern without solving the exercises for you.
- In the repo, `python score.py` shows your scores, `HINTS.md` has three hint levels per exercise, and the README explains setup and how to reset an exercise to redo it.

1. The big picture

Django is a Python web framework. When a request arrives, Django follows the same path every time:

The view then returns a **response** (for an API: JSON plus a status code) that travels back to whoever asked. In the exam, the "whoever" is the unit tests.

Guide A section 10 showed the todo API in Flask next to Django. If Flask stuck better, here is how its pieces map onto Django's (and if you skipped Flask, read the Django column as a summary of this guide):

Flask	Django	Notes
`@app.route("/movies")`	`path("movies", views.f)` in urls.py	Routes live in a list, not in decorators.
a view function	a function in views.py	Takes `request` plus any path parameters.
`request.args`	`request.GET`	Query string values, always strings.
`request.get_json()`	`json.loads(request.body)`	`request.POST` is only for HTML forms.
`jsonify(...)`	`JsonResponse(...)`	Pass `status=201` etc. as a keyword.
SQL / SQLAlchemy	the ORM: `Movie.objects...`	No SQL by hand.
(add-ons)	migrations, admin, test runner built in	`python manage.py ...` runs them.

2. Project layout and commands

A Django **project** (the whole site) contains several **apps** (features). In api-drills the project is `drills/` and each exercise is an app.

File	What it holds
`manage.py`	The command runner. Always run commands from the folder that contains it.
`drills/settings.py`	Configuration. `INSTALLED_APPS` lists the apps; `DATABASES` says SQLite.
`drills/urls.py`	The root URL list. Sends `/ex05/...` to the ex05 app's own `urls.py` with `include`.
`app/models.py`	Model classes, one per database table.
`app/migrations/`	Generated files describing how the tables change over time.
`app/views.py`	View functions.
`app/urls.py`	That app's URL patterns.
`app/tests.py`	Unit tests (any file named `test*.py` is found automatically).

Command	Use it to
`python manage.py test ex05_crud`	run one app's tests (add `.tests.Class.test_name` for one test)
`python manage.py makemigrations`	create migration files after you change a model
`python manage.py migrate`	apply migrations to your local database (tests do this by themselves)
`python manage.py shell`	get a Python prompt with Django loaded, to try ORM queries
`python manage.py runserver`	serve the site at http://127.0.0.1:8000 to try it in a browser
`python manage.py showmigrations`	list migrations and whether they've been applied

3. URLs

Each app has a list called `urlpatterns`. Each entry maps a URL pattern to a view function:

```
from django.urls import path

from . import views

urlpatterns = [
    path("movies", views.movieCollection),            # /api/movies
    path("movies/<int:movieId>", views.movieDetail),  # /api/movies/3
]
```

- **No leading slash** in patterns. The prefix (`api/` here, `ex05/` in the drills) comes from the root `urls.py` via `include(...)`.
- **Path converters** capture part of the URL and pass it to the view as a keyword argument: `<int:movieId>` gives `movieDetail(request, movieId=3)`. If the text isn't an integer, Django answers 404 by itself. Others: `<str:name>`, `<slug:slug>`.
- **The parameter name must match** the view's argument name exactly, or the view crashes with a TypeError.
- **Trailing slashes matter.** `"movies/"` and `"movies"` are different patterns. If a test calls `/api/movies` but your pattern ends in a slash, Django tries to redirect you (status **301**), and for a POST in the dev server it raises an error. Match the URLs in the spec and tests exactly.
- Patterns are checked top to bottom; the first match wins.

> **Practice: ex03_basics**, first half (ping, greet, add). Read `ex03_basics/TASK.md`, then run `python manage.py test ex03_basics` before writing anything, so you see what failure looks like.

4. Views, requests and responses

A view is a function that receives an `HttpRequest` and returns an `HttpResponse`. For APIs you'll mostly return `JsonResponse`.

```
from django.http import JsonResponse

def hello(request):
    name = request.GET.get("name", "world")      # query string: /api/hello?name=Neil
    return JsonResponse({"message": f"Hello, {name}!"})
```

You need	Use
the HTTP method	`request.method` (a string: `"GET"`, `"POST"`, `"PATCH"`, ...)
a query parameter	`request.GET.get("key")` (None if absent; always a string)
the JSON body	`data = json.loads(request.body)` (a dict)
JSON with a status code	`JsonResponse(someDict, status=201)`
a JSON list	`JsonResponse(someList, safe=False)`; without `safe=False` you get a TypeError
an empty response	`HttpResponse(status=204)`
"wrong method"	`HttpResponseNotAllowed(["GET", "POST"])` (status 405) or a JsonResponse with `status=405`
to restrict methods quickly	decorators `@require_GET`, `@require_POST`, `@require_http_methods([...])` from `django.views.decorators.http`

CSRF in one paragraph. Django blocks POST/PATCH/DELETE requests that don't carry a CSRF token (status 403), to protect browser forms. Django's **test client skips this check**, so your tests pass either way. But if you try your API with curl or Postman while running the dev server, you'll get 403 until you put `@csrf_exempt` (from `django.views.decorators.csrf`) above the view. Add it to any view that accepts writes. Real APIs use token authentication instead.

Practice: finish **ex03_basics** (echo and stats). Target: 12/12 in 25 minutes.

5. REST in one page

A REST API exposes **resources** (movies, notes, students) at URLs, and the HTTP method says what to do. Almost every exam-style API has the same two URLs:

Request	Meaning	Success	Common errors
`POST /movies`	create a movie from the JSON body	201 + the created object (with its new id)	400 invalid body
`GET /movies`	list movies (often with ?filters)	200 + an array, in a defined order	-
`GET /movies/3`	read one movie	200 + the object	404 no such id
`PATCH /movies/3`	change **some** fields	200 + the full updated object	404, 400
`PUT /movies/3`	replace the whole object	200	404, 400
`DELETE /movies/3`	delete it	204, empty body	404
any method you don't support	-	-	405 Method Not Allowed

- **409 Conflict** means "valid request, but it clashes with the current state" (e.g. returning a book that's already returned).
- **PATCH vs PUT:** PATCH sends only the fields that change; PUT sends a complete replacement. Exams usually ask for PATCH.
- Read the spec for **ordering** ("ordered by id ascending") and **exact key names** (`first_name`, not `firstName`). Tests compare JSON exactly.

6. Models and the ORM

A model is a class; each attribute is a column. Django creates the `id` primary key for you, numbering rows 1, 2, 3...

```
from django.db import models

class Movie(models.Model):
    title = models.CharField(max_length=200)
    year = models.IntegerField()
    rating = models.FloatField(default=0)
    watched = models.BooleanField(default=False)
    released = models.DateField(null=True, blank=True)   # optional
```

Common fields: `CharField(max_length=...)`, `TextField`, `IntegerField`, `FloatField`, `DecimalField`, `BooleanField`, `DateField`, `DateTimeField`, `ForeignKey(OtherModel, on_delete=models.CASCADE)`.

The ORM, through `Movie.objects`:

```
movie = Movie.objects.create(title="Heat", year=1995)        # INSERT, returns the object
Movie.objects.all()                                           # every row (a QuerySet)
Movie.objects.get(id=3)                                       # exactly one, or raises
Movie.objects.filter(id=999).first()                          # one or None, never raises
Movie.objects.filter(year__gte=1986)                          # WHERE year >= 1986
Movie.objects.filter(title__icontains="alien").order_by("-year")   # chain freely
Movie.objects.exclude(watched=True)
Movie.objects.filter(title="Heat").exists()                   # True / False
Movie.objects.count()

movie.rating = 8.3
movie.save()                                                  # UPDATE one object
Movie.objects.filter(watched=False).update(watched=True)      # UPDATE many, returns count
Movie.objects.filter(year__lt=1980).delete()                  # returns (count, details)
```

Lookup	Meaning	Lookup	Meaning
`year=1995`	equals	`title__iexact="heat"`	equals, ignoring case
`year__gt / __gte`	`>` / `>=`	`title__icontains="x"`	contains, ignoring case
`year__lt / __lte`	`<` / `<=`	`title__startswith="A"`	starts with
`year__in=[1979, 1995]`	is one of	`released__isnull=True`	is NULL

- **Ordering:** `order_by("year")` ascending, `order_by("-year")` descending, `order_by("-rating", "id")` breaks ties by id. Without `order_by` the order isn't guaranteed: always set it when the spec names one.
- **get raises** `Movie.DoesNotExist` when nothing matches. In a view, catch it and return 404. Unhandled, it becomes a 500 (and an ERROR in your tests).
- **QuerySets are lazy:** nothing hits the database until you loop, slice, call `list()`, `count()`, `first()`... So you can build a query up step by step with `if` statements.
- **Aggregates:** `Movie.objects.aggregate(Max("rating"))` gives `{"rating__max": 8.5}`; group-by-and-count is `Movie.objects.values("watched").annotate(total=Count("id"))` (imports from `django.db.models`).

Migrations

The database doesn't read `models.py`; it only knows what the migrations told it. So every model change takes two steps: change the class, then run `python manage.py makemigrations` (and `migrate` for your local database; tests apply migrations by themselves). If you forget, the symptom is an `OperationalError: no such column` (or `no such table`) the moment that model is used. It's a favorite planted bug in debugging tasks.

> **Practice: ex04_orm**. Part B deliberately makes you meet the "no such column" error before you fix it with a migration. Try queries in `python manage.py shell` first if you're unsure.

7. Turning models into JSON

`JsonResponse` can't serialize a model object directly. Write a small helper and use it everywhere:

```
def movieToDict(movie):
    return {"id": movie.id, "title": movie.title, "year": movie.year,
            "rating": movie.rating, "watched": movie.watched}
```

- Shortcut: `from django.forms.models import model_to_dict`; `model_to_dict(movie)` returns every field including `id`. Handy, but a hand-written helper lets you control key names and order.
- **Dates:** `JsonResponse` turns a `date` into `"YYYY-MM-DD"` automatically. Right after `create(released="2026-01-02")` the attribute is still the string you passed in; after reading from the database it's a real `date`. Both come out the same in JSON.
- **Decimals** come out as strings (`"1.50"`), not numbers. If the spec wants a number, convert with `float(...)`.

8. Putting it together: the collection and detail views

Almost every API task is these two views. Here's the collection view in full:

```
import json

from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt

from .models import Movie

@csrf_exempt
def movieCollection(request):
    if request.method == "POST":
        data = json.loads(request.body)
        movie = Movie.objects.create(title=data["title"], year=data["year"])
        return JsonResponse(movieToDict(movie), status=201)
    if request.method == "GET":
        movies = Movie.objects.order_by("id")
        return JsonResponse([movieToDict(m) for m in movies], safe=False)
    return JsonResponse({"error": "method not allowed"}, status=405)
```

The detail view follows a fixed shape. Fill it in yourself in ex05:

```
@csrf_exempt
def movieDetail(request, movieId):
    # 1. reject methods you don't support            -> 405
    # 2. look the row up; if it doesn't exist         -> 404
    # 3. GET:    return it                            -> 200
    # 4. PATCH:  parse the body, set ONLY the keys present, validate, save()  -> 200
    # 5. DELETE: delete it                            -> 204, empty body
```

- **Validate before you change anything.** If a PATCH contains one valid and one invalid field, the spec usually wants a 400 and **nothing** changed.
- **Filters from the query string:** start from a QuerySet and narrow it: `movies = Movie.objects.all()`, then `if year: movies = movies.filter(year=year)`, then order at the end.
- **Booleans in query strings** arrive as the text `"true"`/`"false"`. Compare strings; `bool("false")` is True.

> **Practice: ex05_crud** (40 min), then **ex06_mock_oa** under exam conditions: 45-minute timer, no notes. Check `python score.py ex06` for the visible and "hidden" scores.

9. Testing in Django

Django tests are `unittest` classes with a built-in test client that sends fake requests through your URLs and views:

```
from django.test import TestCase

class MovieApiTests(TestCase):
    def test_create_movie(self):
        response = self.client.post("/api/movies", {"title": "Heat", "year": 1995},
                                    content_type="application/json")
        self.assertEqual(response.status_code, 201)
        self.assertEqual(response.json()["id"], 1)
```

- Every test starts with an **empty database**, and ids restart at 1. Tests can't depend on each other.
- `self.client.get(url, {"key": "value"})` adds a query string. For POST/PATCH send JSON: pass the data and `content_type="application/json"` (for PATCH, pass `json.dumps(data)`).

- `response.status_code`, `response.json()` (fails if the response isn't JSON, e.g. Django's HTML 404 page), `response.content` (raw bytes).
- `setUp(self)` runs before every test in the class: a good place to create shared rows.
- **FAIL vs ERROR:** FAIL means an assertion was false (wrong value). ERROR means an exception was raised (crash). Fix ERRORs first; they often hide FAILs.
- **A good test sits on a boundary:** the exact limit (5000, not 7000), the empty case, the missing id, the case that should be rejected.

> **Practice: ex09_write_tests**. You write the tests; `score.py` swaps in six broken versions of the API and counts how many your tests catch.

10. The debugging playbook

This is the core of the Diagnostics section. Use the same routine every time:

- **Run the tests first** and read the summary: how many fail, and is it FAIL or ERROR?
- **If every test fails the same way**, it's one root cause (often a migration or an import). Fix that first.
- **Take one failing test.** Read its name, then the last lines of its traceback (bottom-up), then the assertion message ("404 != 201" means expected 201, got 404; the order is actual then expected in `assertEqual(response.status_code, 201)`).
- **Find the line** of app code responsible: follow the URL to `urls.py`, then to the view, then to the query.
- **Make the smallest fix**, re-run **everything**, write down the bug, pick the next test.

Symptom	Usual cause	Fix
404 for a URL you think exists	Not in `urlpatterns`, wrong prefix, or an int converter rejecting the value	Compare the test's URL with `urls.py` character by character
301 instead of 200/201	Trailing slash mismatch	Make the pattern match the URL the tests call
405 when you expected success	Method not handled, or the wrong `@require_...` decorator	Check the `request.method` branches
403 (only with curl / runserver)	CSRF protection	`@csrf_exempt` on the view
`TypeError: ... set the safe parameter to False`	Returning a list from `JsonResponse`	`safe=False`
`JSONDecodeError` or an empty dict	Reading `request.POST`, or an empty body	`json.loads(request.body)`
`DoesNotExist` raised in a test	`.get()` without try/except	Catch it and return 404
`OperationalError: no such column/table`	Model changed without a migration	`makemigrations` (then `migrate` locally)
`KeyError: 'x'` in a view	Body lacks a key you indexed directly	`data.get("x")`, or check `"x" in data`
Response right, but a later GET shows the old value	Changed the object but never called `save()`	`obj.save()`
Right items, wrong order	Missing or reversed `order_by`	`order_by("field", "id")`
Filter returns too much / too little	Case sensitivity, `lt` vs `lte`, string vs number	`__iexact`, check the spec's boundary

Symptom	Usual cause	Fix
`response.json()` itself errors in a test	Your view returned HTML (Django's error page)	Look at the status code; it's usually a 404 or 500

Practice: ex07_debug_library (several bugs, some hiding behind others), then **ex08_debug_bank** (harder, starts with a migration bug). 45-minute timer each. Keep a written list of each bug you find.

11. Exam-day plan for the Diagnostics section

When	Do this
Minute 0-3	Read the whole problem statement. Note the URLs, key names, ordering and every status code.
Minute 3-8	Open the test file, `urls.py`, `models.py` and `views.py`. Click **Run Tests** to see the baseline.
Then	If the statement says "do POST first", do. Get one test green early: it proves your URL wiring works.
Every change	Re-run the tests. Points come per test, so each fix is worth something.
Stuck 10 min on one test	Move to another failing test, come back later.
Last 5 minutes	Stop adding features. Make sure everything runs and nothing is half-edited.

Using the built-in assistant well (Django edition). Good, narrow questions:

- "How do I return a JSON list from a Django view?" (answer: `safe=False`)
- "What's the ORM lookup for case-insensitive equality?"
- "What does OperationalError: no such column mean in Django?"
- "Does the problem statement say the list should be ordered by id ascending?"

Avoid "fix the failing tests" or pasting the whole task: it won't solve it, it wastes minutes, and the transcript may be read.

12. Your week, and what comes after

The day-by-day schedule lives in `PLAN.md` in the repo, with a scorecard to fill in. In short: Guide A with ex01 and ex02 first; then this guide with ex03 to ex05; the timed exercises (ex06 Mock OA 1, ex07 and ex08 debugging) next; ex09 for writing tests; and **ex10 Mock OA 2** the day before the real thing, under exam rules. If you fall behind, the must-do list is ex02, ex03, ex05, ex06, ex07 and ex10.

After the OA, to learn Django properly:

- The official tutorial (docs.djangoproject.com, "Writing your first Django app", parts 1-4): templates, the admin site, forms.
- **Django REST Framework** (django-rest-framework.org): serializers, viewsets and routers do the JSON, validation and status codes for you. Most real Django APIs use it. Redo ex05 with it.
- Class-based views, relationships (`ForeignKey`, `select_related`), authentication, and `transaction.atomic()` for multi-step writes (you'll see why in ex08).
- Build something small of your own and deploy it: it's also a better CV line than a tutorial.